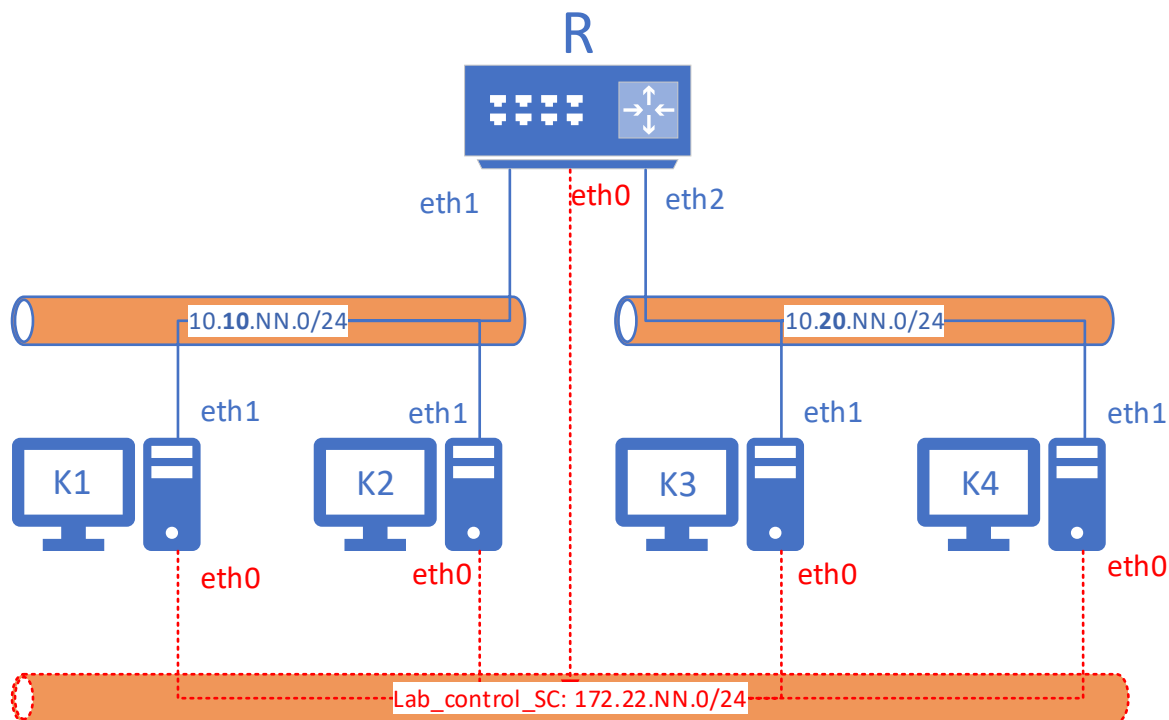


1 Cel ćwiczenia

Celem ćwiczenia jest zapoznanie studentów z podstawami adresacji oraz działania IP.

2 Środowisko laboratoryjne

Dla każdej grupy przygotowano środowisko laboratoryjne, które składa się z 5 kontenerów Docker: K1, K2, K3, K4, R, tak jak przedstawiono na rysunku 1.



K1 oraz K2 to dwa „Komputery” wpięte do sieci lokalnej o adresie 10.10.NN.0/24.

K3 oraz K4 to dwa „Komputery” wpięte do sieci lokalnej o adresie 10.20.NN.0/24.

„Router” R jest wpięty do obu tych sieci.

Wszystkie kontenery korzystają z systemu Ubuntu Server 20.04 i posiadają doinstalowane dodatkowe pakiety programów wykorzystywane w trakcie ćwiczenia.

Wartość NN to numer grupy zadany przez prowadzącego. Oznaczenia grup są dwucyfrowe co ma znaczenie przy wyznaczeniu używanego portu do Logowania (patrz kolejny rozdział). W adresie IP postać 0N zostanie zmieniona na N (np. 10.10.02.0/24 -> 10.10.2.0/24).

Interfejs eth0 służy do podłączenia z siecią zarządzającą (dostęp przez SSH / Internet), natomiast interfejsy eth1 i eth2 do przekazu danych między urządzeniami laboratorium. Interfejs eth0 pozwala na dowolne manipulowanie interfejsami eth1 i eth2 bez ryzyka zerwania połączenia z kontenerem. **Interfejs eth0 nie powinien być w żaden sposób rekonfigurowany** (~~ip a a eth0 ...~~) w trakcie ćwiczenia.

Adresy na interfejsach eth1 i eth2 zostały nadane automatycznie przez Docker. **Należy zwrócić uwagę, że pierwszy adres z każdej sieci jest używany przez serwer/host.** Określenie adresów urządzeń jest częścią ćwiczenia.

2.1 Logowanie

Serwer SSH na kontenerach został wystawiony na konkretny port serwera/hosta o adresie **10.140.0.131** zgodnie z zasadą:

- K1 – 2NN11
- K2 – 2NN12
- K3 – 2NN13
- K4 – 2NN14
- R – 2NN15

Serwer znajduje się w sieci PL-LAB i powinien być osiągalny po połączeniu do VPN PL-LAB.

Logując się przez SSH należy użyć **użytkownika „root”** i **hasła „rootNN”**.

Przykładowo, grupa NN=88 łączy się na Komputer K1 poleceniem:

```
ssh root@10.140.0.131 -p 28811
```

a na Ruter R poleceniem:

```
ssh root@10.140.0.131 -p 28815
```

UWAGA: Komputer za pomocą którego wykonywane jest ćwiczenie, będzie w tej instrukcji nazywany jako: **Komputer Studenta**

2.2 Przydatne komendy

2.2.1 Podstawowe komendy

ps -au	wypisanie listy działających procesów
history	wypisuje listę wszystkich wpisanych komend
<tab>	Wciśnięcie klawisza Tabulacji powoduje uzupełnienie wpisywanego tekstu (np. komendy, folderu, czy nazwy pliku)
<Ctrl+c>	Wciśnięcie kombinacji <Ctrl+c> powoduje zatrzymanie działającego procesu
<strzałki>	Strzałki w górę i w dół przewijają ostatnio używane komendy.

Komendy niepotrzebne do wykonania ćwiczenia (ale tak podstawowe, że musiały się tu znaleźć):	
ls	wypisanie zawartości katalogu.
ls -lh	wypisanie zawartości katalogu w postaci listy wraz z właściwościami plików
pwd	Wypisanie ścieżki bezwzględnej katalogu, w którym aktualnie się znajdujemy
cat <plik>	wypisanie zawartości pliku <plik>
cp <plik1> <plik2>	Skopiowanie <plik1> do <plik2>
cd <path>	przejdzie do katalogu opisanego w ścieżce <path>, ścieżka może być bezwzględna (rozpoczęta znakiem „/”) lub względna. • ścieżka bezwzględna to cała ścieżka, od poziomu „/”, np. komenda <code>cd /root/my_data</code> spowoduje przejście do katalogu <code>/root/my_data</code>, niezależnie od tego, w jakim katalogu komenda została wykonana. • Ścieżka względna to ścieżka wyznaczona od poziomu katalogu, w którym aktualnie się znajdujemy, np. <code>cd my_data</code> wywołane w katalogu <code>/root/</code> spowoduje przejście do katalogu <code>/root/my_data</code>, ale wywołana w innym katalogu zwróci błąd „No such file or directory”. <code>cd ..</code> - przejście poziom wyżej w strukturze katalogów

2.2.2 ping

Program „ping” pozwala na wysłanie zapytania ICMP Echo Request na podany adres IP. Jeśli zapytanie zostanie odebrane przez wywoływany maszynę, powinna ona odpowiedzieć przy pomocy wiadomości ICMP Echo Reply, chyba że została skonfigurowana, aby tego nie robić.

- `ping <adres_IP>` - wysłanie pingu na <adres_IP>

W poleceniu ping nie podajemy maski adresu IP.

2.2.3 ip a (ip address)

Polecenia z rodziny `ip` zastępują przestarzałe `ifconfig`. Służą do przeglądania i modyfikowania adresacji oraz stanu interfejsów sieciowych. W systemie Windows analogiczna komenda do podglądu to `ipconfig`.

- `ip a (ddress)` – wypisuje informacje o adresach na wszystkich interfejsach
- `ip -br a` – tylko najważniejsze informacje
- `ip -4 a, ip -6 a` – tylko IPv4 lub IPv6
- `ip a s(how) dev <interfejs>` – informacje o danym interfejsie
- `ip a a(dd) <adres>/<maska> dev <interfejs>` – nadanie adresu IP na interfejsie.
Podajemy maskę w notacji prefixu, np. /24.
 - o Przykład: `ip a a 10.11.12.13/24 dev eth1`
- `ip a d(el) <adres>/<maska> dev <interfejs>` – usunięcie adresu z interfejsu.
- `ip a flush dev <interfejs>` – wyczyszczenie wszystkich adresów z interfejsu
- `ip link set <interfejs> up / down` – włączenie/wyłączenie interfejsu

2.2.4 ip r (ip route)

`ip route` służy do manipulowania tablicą routingu danej maszyny (wybór trasy, bramki, metryki). Wpisy w tej tablicy są dodawane automatycznie na podstawie adresów nadanych interfejsom, wynikają z konfiguracji, lub mogą być dodawane ręcznie. W systemie Windows odpowiednik podglądu to `route print`.

- `ip r(oute)` – wypisuje tablicę routingu
- `route -n` – stara wersja wypisywania tablicy – dużo czytelniejsza
- `ip -4 r, ip -6 r` – tylko IPv4 / IPv6
- `ip r a(dd) <sieć>/<maska> via <adres_bramy>` – dodanie trasy do sieci przez bramę.
 - o Przykład: `ip r a 10.11.13.0/24 via 10.11.12.1`
- `ip r a default via <adres_bramy> dev <interfejs>` – ustawienie trasy domyślnej (*default gateway*):
 - o Przykład: `ip r a default via 10.11.12.1`
- `ip r d(el) <sieć>/<maska> via <adres_bramy>` – usunięcie wpisu z tablicy routingu.
- Opcjonalnie można dodać interfejs i metrykę trasy:
 - o `ip r a <sieć>/<maska> via <adres_bramy> dev <interfejs> metric <metryka>` (niższa metryka = wyższy priorytet)

*Podawanie interfejsu do `ip r` jest zazwyczaj przydatne tylko dla nietypowych konfiguracji (dwa interfejsy w tej samej sieci, brama onlink <https://unix.stackexchange.com/questions/579087/whats-the-meaning-of-linkdown-and-onlink-in-linux-route>, ...) – zazwyczaj adres bramy definiuje jednoznacznie interfejs, którym należy wysłać ruch. Natomiast podawanie metryki przy trasie już jest częstsze - gdy mamy różne drogi dojścia do danej sieci, w szczególności wyjście do Internetu (*default gateway*) i jedna ścieżka jest preferowana, a druga jako backup (na awarie interfejsu lub bramy; awaria w dalszej części ścieżki nie będzie skutkować przełączeniem drogi).*

2.2.5 tcpdump

Polecenie „tcpdump” służy do obserwacji ruchu na interfejsie sieciowym.

- `tcpdump -i <interfejs>` - przechwytuje ramki odbierane na <interfejsie> i wyświetla na ekran podstawowe informacje o tych ramach
 - dodanie flagi `-n` - bez nazw (preferowane)
 - dodanie flagi `-v` - wyświetla bardziej szczegółowe dane (czasami przydatne)
 - flagi można łączyć
- Jeśli szukamy jakiegoś konkretnego ruchu, to warto stosować filtry. Można filtrować pod kątem różnych parametrów, można łączyć filtry, można stosować zaprzeczenia, np.:
 - `tcpdump -i eth0 -n port 80`
 - `tcpdump -i eth0 -n arp`
 - `tcpdump -i eth0 -n icmp and host 10.1.1.1`
 - `tcpdump -i eth0 -n host 10.1.1.1 and not port 22`
 - Ściąga filtrów: <https://www.tcpdump.org/manpages/pcap-filter.7.html>

2.2.6 ip n(eighbour) / arp

Polecenie „ip n” oraz „arp” wyświetlają tablicę ARP – odwzorowanie adresów L2/L3.

- `ip n / arp -n` - tablica ARP bez nazw (zalecana opcja)

2.2.7 ss

Program „ss” (z pakietu *iproute2*) wyświetla listę gniazd/połączeń sieciowych i zastępuje przestarzałe `netstat`. Warto zapamiętać zestaw flag „tulpn” (np. jako „tulipan”).

- `ss -tulpn` – pokazuje **tylko nasłuchujące** (`-l`) gniazda TCP/UDP (`-t`, `-u`) wraz z PID/programem (`-p`); adresy i porty w formie numerycznej (`-n`).
- `ss -tupn` – pokazuje **tylko aktywne** gniazda (tj. istniejące połączenia) TCP/UDP (`-t`, `-u`) wraz z PID/programem (`-p`); adresy i porty w formie numerycznej (`-n`).
- `ss -tupna` – pokazuje **wszystkie** gniazda TCP/UDP ...

3 Zadania do wykonania

W tej części przedstawione zostaną zadania do wykonania.

3.1 Zespół projektowy

Imię	nazwisko	nr indeksu	adres otrzymany z VPN
Kinga	Konieczna	337072	10.141.6.3
Jan	Czechowski	337066	
Tymon	Zadara	337086	

Adres otrzymany z VPN może się zmieniać po restarcie podłączenia. Otrzymany adres można zobaczyć przy pomocy opcji „pokaż status” programu OpenVPN, lub bezpośrednio w terminalu swojego komputera.

3.2 Połączenie z siecią PL-LAB

W celu zapewnienia połączenia z siecią laboratoryjną wykorzystano dostęp przez bramkę VPN. Po podłączeniu do bramki VPN, Komputerowi Studenta powinien zostać przyznany adres z puli 10.141.6.0-10.141.6.255 z maską 9, który umożliwi dostęp do sieci 10.128.0.0/9.

3.2.1.1 Jaki zakres adresów obejmują ww. sieci? Proszę uzupełnić tabelkę:

Sieć 10.141.6.0/24	
adres sieci	10.141.6.0
najmniejszy możliwy adres dostępny dla hosta	10.141.6.1
największy możliwy adres dostępny dla hosta	10.141.6.254
adres broadcast	10.141.6.255

Sieć 10.128.0.0/9	
adres sieci	10.128.0.0
najmniejszy możliwy adres dostępny dla hosta	10.128.0.1
największy możliwy adres dostępny dla hosta	10.255.255.254
adres broadcast	10.255.255.255

3.2.1.2 Do czego służy adres broadcast?

Adres broadcast, czyli adres rozgłoszeniowy służy do wysłania danych jednocześnie do wszystkich urządzeń w sieci lokalnej (do wszystkich hostów).

3.2.2 Weryfikacja otrzymanych adresów i tablicy routingu

Proszę zamieścić zrzuty ekranu z terminala Komputera Studenta, które potwierdzają otrzymanie adresu z puli 10.141.6.0-10.141.6.255 oraz wpisu do tablicy routingu do sieci 10.128.0.0/9. W przypadku większej ilości tekstu na zrzucie ekranu, proszę zaznaczyć fragment bezpośrednio odnoszący się do polecenia.

Zrzut ekranu – adres ip

```
Unknown adapter OpenVPN TAP-Windows6:

Connection-specific DNS Suffix  . : 
Link-local IPv6 Address . . . . . : fe80::81d:41b1:5e81:fe5e%7
IPv4 Address. . . . . : 10.141.6.3
Subnet Mask . . . . . : 255.128.0.0
Default Gateway . . . . . :
```

Zrzut ekranu – tablica routingu

```
IPv4 Route Table
=====
Active Routes:
Network Destination        Netmask          Gateway          Interface        Metric
0.0.0.0                    0.0.0.0          192.168.114.254  192.168.114.172   25
10.128.0.0                 255.128.0.0      On-link          10.141.6.3        281
10.141.6.3                 255.255.255.255  On-link          10.141.6.3        281
10.255.255.255             255.255.255.255  On-link          10.141.6.3        281
127.0.0.0                  255.0.0.0        On-link          127.0.0.1         331
127.0.0.1                  255.255.255.255  On-link          127.0.0.1         331
127.255.255.255           255.255.255.255  On-link          127.0.0.1         331
```

Wyświetlenie tablicy routingu w Windows: *route print*

3.2.3 Połączenie na kontenery oraz między kontenerami

Należy zalogować się na kontenery K1 i K3, oraz sprawdzić ich konfigurację sieciową.

	K1	K3
eth0:	172.22.7.4/24	172.22.7.2/24
eth1:	10.10.7.3/24	10.20.7.2/24

```
root@07_K1:~# ip -br a
lo                UNKNOWN        127.0.0.1/8 ::1/128
eth0@if1123       UP              172.22.7.4/24
eth1@if1128       UP              10.10.7.3/24
```

```
root@07_K3:~# ip -br a
lo                UNKNOWN        127.0.0.1/8 ::1/128
eth0@if1119       UP              172.22.7.2/24
eth1@if1124       UP              10.20.7.2/24
```

Logowanie na kontenery zgodnie z pkt 2.1, w skrócie:

```
ssh root@10.140.0.131 -p port_num
```

port_num:

- K1 – 2NN11
- K2 – 2NN12
- K3 – 2NN13
- K4 – 2NN14
- R – 2NN15

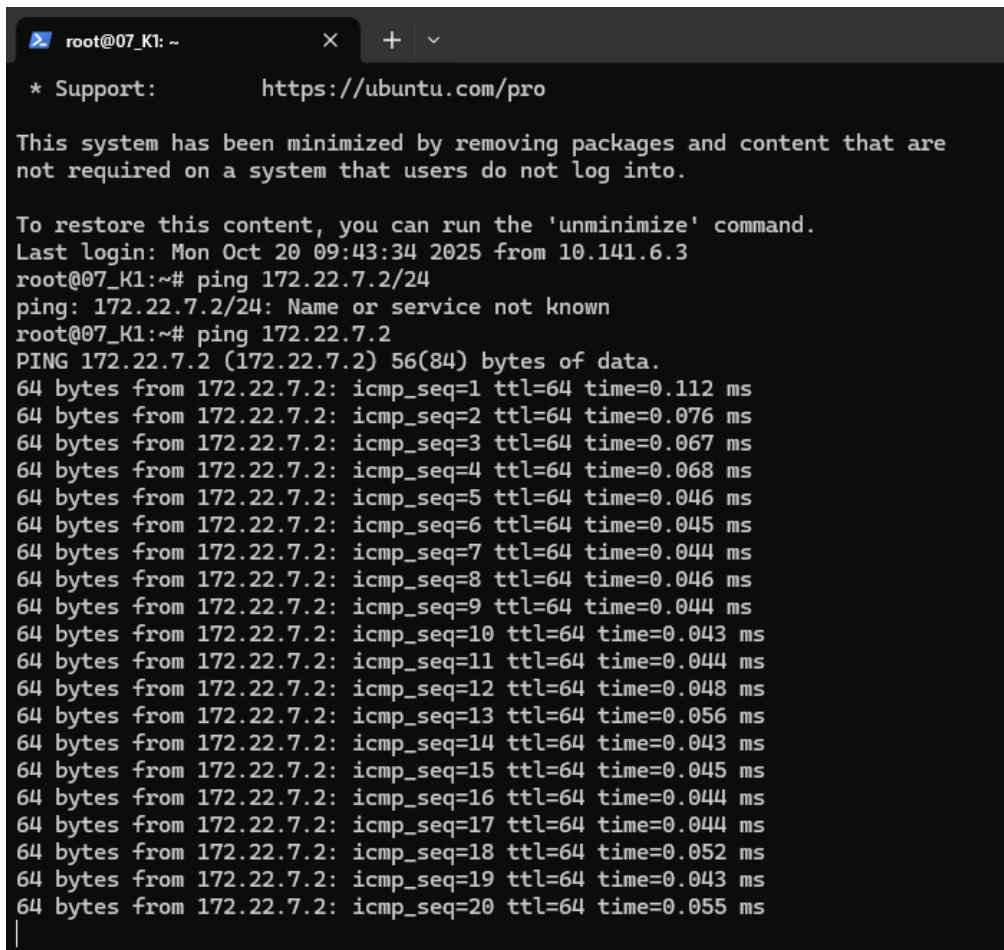
Należy spróbować spingować K3 z K1 używając jako adresu docelowego

- a) adresu K3/eth0
- b) adresu K3/eth1

```
ping <adres_IP> - wysłanie pinga na <adres_IP>
```

3.2.3.1 Jaki jest wynik działania polecenia ping w obu przypadkach?

Ping na adres K3/eth0 - Zrzut ekranu z K1



```
root@07_K1: ~
* Support: https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Oct 20 09:43:34 2025 from 10.141.6.3
root@07_K1:~# ping 172.22.7.2/24
ping: 172.22.7.2/24: Name or service not known
root@07_K1:~# ping 172.22.7.2
PING 172.22.7.2 (172.22.7.2) 56(84) bytes of data.
64 bytes from 172.22.7.2: icmp_seq=1 ttl=64 time=0.112 ms
64 bytes from 172.22.7.2: icmp_seq=2 ttl=64 time=0.076 ms
64 bytes from 172.22.7.2: icmp_seq=3 ttl=64 time=0.067 ms
64 bytes from 172.22.7.2: icmp_seq=4 ttl=64 time=0.068 ms
64 bytes from 172.22.7.2: icmp_seq=5 ttl=64 time=0.046 ms
64 bytes from 172.22.7.2: icmp_seq=6 ttl=64 time=0.045 ms
64 bytes from 172.22.7.2: icmp_seq=7 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=8 ttl=64 time=0.046 ms
64 bytes from 172.22.7.2: icmp_seq=9 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=10 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=11 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=12 ttl=64 time=0.048 ms
64 bytes from 172.22.7.2: icmp_seq=13 ttl=64 time=0.056 ms
64 bytes from 172.22.7.2: icmp_seq=14 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=15 ttl=64 time=0.045 ms
64 bytes from 172.22.7.2: icmp_seq=16 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=17 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=18 ttl=64 time=0.052 ms
64 bytes from 172.22.7.2: icmp_seq=19 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=20 ttl=64 time=0.055 ms
```

Ping na adres K3/eth1 - Zrzut ekranu z K1

```

root@07_K1: ~
64 bytes from 172.22.7.2: icmp_seq=7 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=8 ttl=64 time=0.046 ms
64 bytes from 172.22.7.2: icmp_seq=9 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=10 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=11 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=12 ttl=64 time=0.048 ms
64 bytes from 172.22.7.2: icmp_seq=13 ttl=64 time=0.056 ms
64 bytes from 172.22.7.2: icmp_seq=14 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=15 ttl=64 time=0.045 ms
64 bytes from 172.22.7.2: icmp_seq=16 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=17 ttl=64 time=0.044 ms
64 bytes from 172.22.7.2: icmp_seq=18 ttl=64 time=0.052 ms
64 bytes from 172.22.7.2: icmp_seq=19 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=20 ttl=64 time=0.055 ms
64 bytes from 172.22.7.2: icmp_seq=21 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=22 ttl=64 time=0.046 ms
64 bytes from 172.22.7.2: icmp_seq=23 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=24 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=25 ttl=64 time=0.043 ms
64 bytes from 172.22.7.2: icmp_seq=26 ttl=64 time=0.044 ms
ex64 bytes from 172.22.7.2: icmp_seq=27 ttl=64 time=0.043 ms
it
64 bytes from 172.22.7.2: icmp_seq=28 ttl=64 time=0.043 ms
q64 bytes from 172.22.7.2: icmp_seq=29 ttl=64 time=0.046 ms
64 bytes from 172.22.7.2: icmp_seq=30 ttl=64 time=0.046 ms
^C
--- 172.22.7.2 ping statistics ---
30 packets transmitted, 30 received, 0% packet loss, time 29692ms
rtt min/avg/max/mdev = 0.043/0.050/0.112/0.014 ms
root@07_K1:~# ping 10.20.7.2
PING 10.20.7.2 (10.20.7.2) 56(84) bytes of data.
|

```

3.2.3.2 Dlaczego otrzymano inny wynik?

Odp. Otrzymano inny wynik (na eth0 doszedł ping, a na eth1 nie), ponieważ komputer K1 jest połączony z eth0 komputera K3 przez Lab_control_SC, a eth1 komputera K3 nie, więc ping nie dociera.

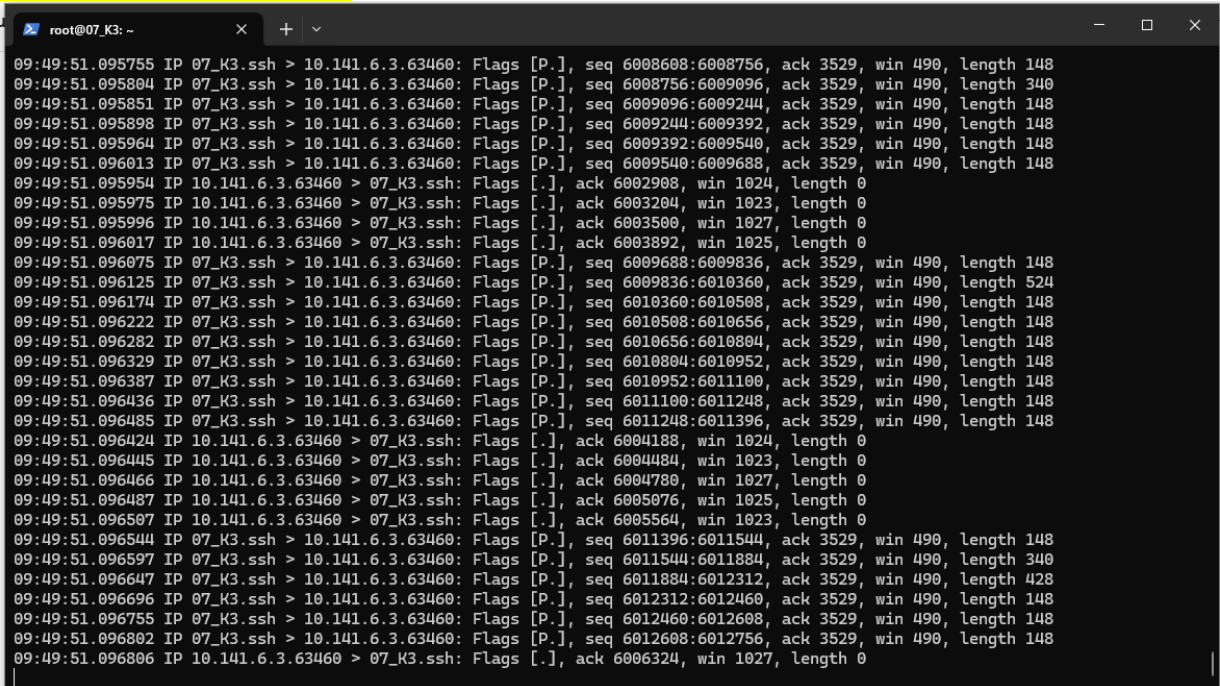
3.3 Obserwowanie ruchu

To zadanie ma na celu zapoznanie Studentów z komendą `tcpdump`.

Na kontenerze K3 należy uruchomić obserwowanie ruchu na interfejsie **eth0**. Wystarczy kilka sekund obserwacji.

3.3.1.1 Wynik obserwacji:

Ruch na K3/eth0 - Zrzut ekranu



```
root@07_K3: ~  
09:49:51.095755 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6008608:6008756, ack 3529, win 490, length 148  
09:49:51.095804 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6008756:6009096, ack 3529, win 490, length 340  
09:49:51.095851 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6009096:6009244, ack 3529, win 490, length 148  
09:49:51.095898 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6009244:6009392, ack 3529, win 490, length 148  
09:49:51.095964 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6009392:6009540, ack 3529, win 490, length 148  
09:49:51.096013 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6009540:6009688, ack 3529, win 490, length 148  
09:49:51.095954 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6002908, win 1024, length 0  
09:49:51.095975 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6003204, win 1023, length 0  
09:49:51.095996 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6003500, win 1027, length 0  
09:49:51.096017 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6003892, win 1025, length 0  
09:49:51.096075 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6009688:6009836, ack 3529, win 490, length 148  
09:49:51.096125 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6009836:6010360, ack 3529, win 490, length 524  
09:49:51.096174 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6010360:6010508, ack 3529, win 490, length 148  
09:49:51.096222 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6010508:6010656, ack 3529, win 490, length 148  
09:49:51.096282 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6010656:6010804, ack 3529, win 490, length 148  
09:49:51.096329 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6010804:6010952, ack 3529, win 490, length 148  
09:49:51.096387 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6010952:6011100, ack 3529, win 490, length 148  
09:49:51.096436 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6011100:6011248, ack 3529, win 490, length 148  
09:49:51.096485 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6011248:6011396, ack 3529, win 490, length 148  
09:49:51.096424 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6004188, win 1024, length 0  
09:49:51.096445 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6004484, win 1023, length 0  
09:49:51.096466 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6004780, win 1027, length 0  
09:49:51.096487 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6005076, win 1025, length 0  
09:49:51.096507 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6005564, win 1023, length 0  
09:49:51.096544 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6011396:6011544, ack 3529, win 490, length 148  
09:49:51.096597 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6011544:6011884, ack 3529, win 490, length 340  
09:49:51.096647 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6011884:6012312, ack 3529, win 490, length 428  
09:49:51.096696 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6012312:6012460, ack 3529, win 490, length 148  
09:49:51.096755 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6012460:6012608, ack 3529, win 490, length 148  
09:49:51.096802 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P.], seq 6012608:6012756, ack 3529, win 490, length 148  
09:49:51.096806 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 6006324, win 1027, length 0
```

`tcpdump -i <nazwa_interfejsu>`

3.3.1.2 Co to za ruch i skąd to wiemy?

Odp: To ruch SSH – komunikacja pomiędzy użytkownikiem (10.141.6.3) a kontenerem K3, który działa jako serwer SSH.

Skąd to wiemy?

- Port **22/tcp** (ssh) wskazuje na połączenie SSH.
- Adres 10.141.6.3 to nasz komputer.
- Numer portu 63460 to port tymczasowy klienta.
- Flags [P.] to normalne dane sesji SSH – przesyłanie poleceń lub danych.
- Flags [..] to potwierdzenia (ACK) – czyli że pakiet został odebrany.

3.4 Podstawowa konfiguracja routingu

To zadanie ma na celu zapoznanie Studentów z podstawową ideą routingu.

Z jednego komputera należy mieć uruchomione 3 połączenia SSH na K1 oraz 2 połączenia SSH na K3 (razem 5 terminali). Na 4 terminalach należy uruchomić obserwowanie ruchu na interfejsie eth0 i eth1 kontenerów K1 i K3 przy pomocy `tcpdump`.

Uwaga: na interfejsach eth0 będzie bardzo dużo ruchu związanego z połączeniem SSH, więc w tcpdump można użyć filtra który wytnie ruch na port 22, np.: **tcpdump -i eth0 -n not port 22**

Z piątego terminala uruchomić ping z K1 na K3 (używając jako adresu docelowego adresu K3/eth1).

3.4.1.1 Czy jakiś ruch jest widoczny na którymkolwiek z interfejsów?

Zrzut ekranu – 4 terminale z tcpdump na interfejsach eth0 i eth1 na K1 i K3

```
root@07_K1:~# tcpdump -i eth0 -n not port 22
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, Link-type EN10MB (Ethernet), capture size 262144 bytes
09:53:39.488983 ARP, Request who-has 172.22.7.4 tell 172.22.7.1, length 28
09:53:39.488983 ARP, Reply 172.22.7.4 is-at 12:65:48:e2:a7:35, length 28
09:54:41.437858 ARP, Request who-has 172.22.7.4 tell 172.22.7.1, length 28
09:54:41.437858 ARP, Reply 172.22.7.4 is-at 12:65:48:e2:a7:35, length 28
09:55:24.956981 ARP, Request who-has 172.22.7.4 tell 172.22.7.1, length 28
09:55:24.957047 ARP, Reply 172.22.7.4 is-at 12:65:48:e2:a7:35, length 28
09:55:30.076922 ARP, Request who-has 172.22.7.1 tell 172.22.7.4, length 28
09:55:30.077822 ARP, Reply 172.22.7.1 is-at 3e:ed:85:1a:ae:9d, length 28
09:56:23.325888 ARP, Request who-has 172.22.7.4 tell 172.22.7.1, length 28
09:56:23.325893 ARP, Reply 172.22.7.4 is-at 12:65:48:e2:a7:35, length 28
09:57:24.233552 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 1, length 64
09:57:25.277826 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 2, length 64
09:57:26.389982 ARP, Request who-has 172.22.7.4 tell 172.22.7.1, length 28
09:57:26.391818 ARP, Reply 172.22.7.4 is-at 12:65:48:e2:a7:35, length 28
09:57:26.391832 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 3, length 64
09:57:27.325828 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 4, length 64
09:57:28.349852 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 5, length 64
09:57:29.373838 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 6, length 64
09:57:30.397826 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 7, length 64
09:57:31.421827 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 8, length 64
09:57:32.445825 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 9, length 64
09:57:33.469822 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 10, length 64
09:57:34.493830 IP 172.22.7.4 > 10.28.7.2: ICMP echo request, id 197, seq 11, length 64

root@10_K3:~# tcpdump -i eth1 -n not port 22
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, Link-type EN10MB (Ethernet), capture size 262144 bytes
Length 244
09:50:10.358421 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 78498312, win 1024, length 0
09:50:10.358441 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 78498624, win 1023, length 0
09:50:10.358460 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 78498936, win 1022, length 0
09:50:10.358479 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [P..], seq 45901:45937, ack 78498936, win 1022, length 36
09:50:10.358503 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P..], seq 78507616:78508148, ack 45937, win 490, length 532
09:50:10.358549 IP 07_K3.ssh > 10.141.6.3.63460: Flags [P..], seq 78508148:78508394, ack 45937, win 490, length 156
09:50:10.358522 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 78499372, win 1027, length 0
09:50:10.358542 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 78509160, win 1023, length 0
09:50:10.358561 IP 10.141.6.3.63460 > 07_K3.ssh: Flags [..], ack 78509780, win 1027, length 0
^C
585880 packets captured
587424 packets received by filter
1620 packets dropped by kernel
root@07_K1:~# tcpdump -i eth0 -n not port 22
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, Link-type EN10MB (Ethernet), capture size 262144 bytes
09:53:12.868975 ARP, Request who-has 172.22.7.2 tell 172.22.7.1, length 28
09:53:12.861014 ARP, Reply 172.22.7.2 is-at e6:8e:ad:22:77:97, length 28
09:54:04.573291 ARP, Request who-has 172.22.7.2 tell 172.22.7.1, length 28
09:54:04.573021 ARP, Reply 172.22.7.2 is-at e6:8e:ad:22:77:97, length 28
09:55:35.196979 ARP, Request who-has 172.22.7.1 tell 172.22.7.2, length 28
09:55:35.196980 ARP, Reply 172.22.7.1 is-at 3e:ed:85:1a:ae:9d, length 28
09:55:35.197019 ARP, Request who-has 172.22.7.2 tell 172.22.7.1, length 28
09:55:35.197037 ARP, Reply 172.22.7.2 is-at 3e:ed:85:1a:ae:9d, length 28
```

Odp:

W prawym górnym rogu, czyli na eth0 komputera K1 widać ciągły ruch (ICMP oraz ARP), a na eth0 komputera K3 widać protokoły ARP. Pakiety ICMP docierają do eth0, ponieważ system wysyła pakiet przez eth0, ponieważ z jego punktu widzenia tamtędy prowadzi trasa do adresu K3/eth1.

Zatrzymać program ping (Ctrl+c).

3.4.1.2 Sprawdzić tablicę routingu na K1.

Zrzut ekranu – tablica routingu K1

```
root@07_K1:~# route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 172.22.7.1 0.0.0.0 UG 0 0 0 eth0
10.10.7.0 0.0.0.0 255.255.255.0 U 0 0 0 eth1
172.22.7.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
root@07_K1:~#
```

route -n - wypisuje tablicę routingu

3.4.1.3 Na podstawie jakiego pola w nagłówku pakietu IP, pakiet jest przesyłany w odpowiednie miejsce?

Odp: Pakiet IP jest przesyłany na podstawie pola: Destination - adres docelowy w nagłówku pakietu IP. Na jego podstawie system operacyjny sprawdza tablicę routingu i wybiera odpowiedni interfejs lub bramę, do której ma wysłać pakiet.

Z otrzymanej tablicy routingu wynika, że:

- ruch kierowany do sieci 172.22.NN.0/24* jest kierowany na interfejs eth0 (brak bramy – Gateway=0.0.0.0),
- ruch kierowany do sieci 10.10.NN.0/24 jest kierowany na interfejs eth1 (brak bramy – Gateway=0.0.0.0),
- Ruch do sieci 0.0.0.0/0 jest kierowany na interfejs eth0 na bramę 172.22.NN.1.

Sieć 0.0.0.0/0 oznacza „wszystkie inne adresy”.

*maska jest w tym przypadku zapisana w postaci 4 oktetów i wynosi 255.255.255.0, co odpowiada masce o długości 24 bitów.

3.4.1.4 Dlaczego jedne wpisy posiadają bramę a inne nie?

Odp: Wpisy bez bramy oznaczają sieci lokalne, osiągalne bezpośrednio przez interfejs. Wpisy z bramą wskazują na konieczność przekazania pakietu do routera pośredniczącego, który dalej skieruje dalej pakiety.

3.4.1.5 Gdzie kierowany jest ruch adresowany na adres K3/eth1?

Odp: Na default, czyli 0.0.0.0.

```
root@07_K1:~# route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          172.22.7.1     0.0.0.0         UG    0      0      0 eth0
10.10.7.0        0.0.0.0        255.255.255.0   U     0      0      0 eth1
172.22.7.0       0.0.0.0        255.255.255.0   U     0      0      0 eth0
root@07_K1:~# ip r
default via 172.22.7.1 dev eth0
10.10.7.0/24 dev eth1 proto kernel scope link src 10.10.7.3
172.22.7.0/24 dev eth0 proto kernel scope link src 172.22.7.4
root@07_K1:~#
```

Na K1 należy dodać wpis do tablicy routingu, który skieruje pakiety adresowane do sieci 10.20.NN.0/24 na adres R/eth1.

```
root@07_R:~# ip -br a
lo                UNKNOWN        127.0.0.1/8 ::1/128
eth1@if1122       UP             10.10.7.254/24
eth2@if1127       UP             10.20.7.254/24
eth0@if1129       UP             172.22.7.6/24
root@07_R:~# |
```

ip r a <sieć>/<maska> via <adres_bramy> - dodaje wpis do tablicy routingu, który oznacza: „Pakiety kierowane do sieci <sieć>/<maska> przesyłaj na adres <adres_bramy>”.

3.4.1.6 Po dodaniu odpowiedniego wpisu, należy udokumentować zmiany w tablicy routingu.

Zrzut ekranu – tablica routingu K1

```
root@07_K1:~# ip r a 10.20.7.0/24 via 10.10.7.254
root@07_K1:~# route -n
Kernel IP routing table
Destination        Gateway            Genmask           Flags Metric Ref    Use Iface
0.0.0.0            172.22.7.1        0.0.0.0           UG        0      0        0 eth0
10.10.7.0          0.0.0.0           255.255.255.0     U        0      0        0 eth1
10.20.7.0          10.10.7.254       255.255.255.0     UG        0      0        0 eth1
172.22.7.0         0.0.0.0           255.255.255.0     U        0      0        0 eth0
root@07_K1:~#
```

Pojawił się 10.10.7.0.

`route -n` - wypisuje tablicę routingu bez nazw (zalecana opcja)

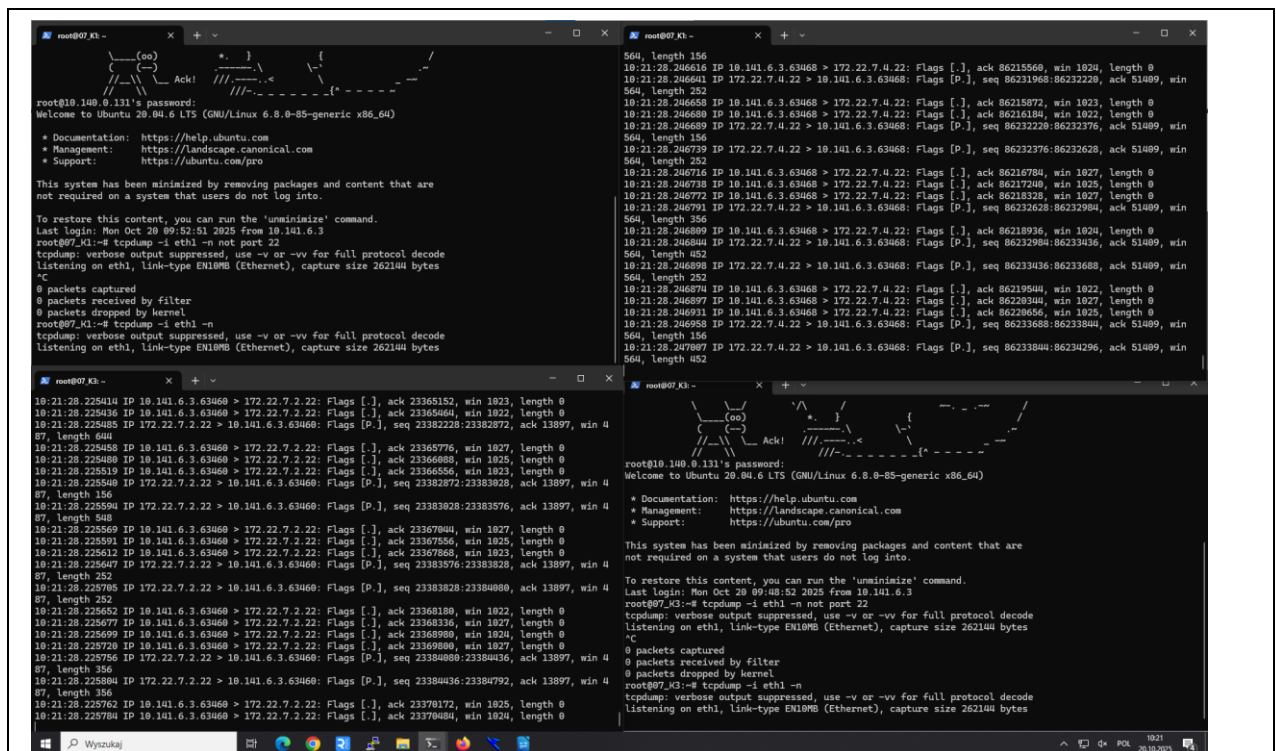
3.4.1.7 Czemu bramą dla K1 jest R/eth1 a nie R/eth2?

Odp: Ponieważ interfejs R/eth1 znajduje się w tej samej sieci co K1, a więc jest osiągalny bezpośrednio z K1. Interfejs R/eth2 należy do innej podsieci (10.20.7.0/24), która nie jest bezpośrednio dostępna z K1, więc nie może być użyta jako brama bez pośredniego routera.

3.4.1.8 Czy po dodaniu na K1 wpisu do tablicy routingu jakiś ruch jest widoczny na którymkolwiek z interfejsów?

Zrzut ekranu – 4 terminale z tcpdump na interfejsach eth0 i eth1 na K1 i K3

The screenshot displays four terminal windows. The top-left window shows a network diagram with hosts K1, K3, and R. The top-right window shows a list of ICMP echo requests from K1 to R. The bottom-left window shows a list of ARP requests and replies between K1 and K3. The bottom-right window shows a list of ARP requests and replies between K1 and R.



Odp: Ruch jest widoczny na interfejsie eth0 na K1 i K3 na porcie 22, ponieważ przy zastosowaniu filtra not port 22 nie było pakietów, a po usunięciu zastosowanego filtra pakiety się pojawiły.

`tcpdump -i <nazwa_interfejsu> -n`

3.4.1.9 Dlaczego, skoro do K3 docierają zapytania ICMP echo request, to nie odpowiada on na nie? [podchwytliwe] (podpowiedź: K3 odpowiada, ale czy kieruje odpowiedzi w dobre miejsce?)

Odp: K3 odpowiada na zapytania ICMP echo request, lecz kieruje je na zły interfejs.

Należy dokonać odpowiedniej zmiany w konfiguracji K3, tak by ping zaczął poprawnie działać.

3.4.1.10 Udokumentować zrzutem ekranu.

Odp: Skierowaliśmy odpowiedzi na interfejs eth0 i komputer K3 zaczął poprawnie odpowiadać.

Zrzut ekranu – 4 terminale z tcpdump na interfejsach eth0 i eth1 na K1 i K3 oraz terminal z programem ping

```
root@07_K1:~#
10:37:46.567604 ARP, Request who-has 10.10.7.254 tell 10.10.7.3, length 28
10:37:46.567658 ARP, Reply 10.10.7.254 is-at 76:f2:be:f6:el:99, length 28
10:37:46.567662 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 1, length 64
10:37:47.613831 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 2, length 64
10:37:48.637829 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 3, length 64
10:37:49.644133 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 4, length 64
10:37:50.684997 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 5, length 64
10:37:51.799919 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 6, length 64
10:37:52.723868 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 7, length 64
10:37:53.757807 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 8, length 64
10:37:54.781842 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 9, length 64
10:37:55.885839 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 10, length 64
10:37:56.829180 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 11, length 64
10:37:57.853931 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 12, length 64
10:37:58.877829 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 13, length 64
10:37:59.901850 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 14, length 64
10:38:00.925822 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 15, length 64
10:38:01.949885 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 16, length 64
10:38:02.973836 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 17, length 64
10:38:03.997824 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 18, length 64
10:38:05.029883 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 19, length 64
10:38:06.049588 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 20, length 64
10:38:07.068998 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 21, length 64
10:38:08.097828 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 22, length 64
10:38:09.117833 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 23, length 64
10:38:10.141804 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 24, length 64

root@07_K1:~#
10:37:56.829124 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 11, length 64
10:37:57.852987 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:37:57.853084 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 12, length 64
10:37:58.877846 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 13, length 64
10:37:58.877872 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:37:59.901883 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 14, length 64
10:38:00.924966 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:00.925047 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 15, length 64
10:38:01.949879 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:01.949880 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:02.972981 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:02.973051 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 17, length 64
10:38:03.996991 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:03.997048 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 18, length 64
10:38:05.021017 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 19, length 64
10:38:05.021049 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:06.044953 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:06.045091 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 20, length 64
10:38:07.068976 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:07.069038 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 21, length 64
10:38:08.093865 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 22, length 64
10:38:08.093895 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:09.116949 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:09.117064 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 23, length 64
10:38:10.140966 ARP, Request who-has 10.10.7.3 tell 10.20.7.2, length 28
10:38:10.141838 IP 10.10.7.3 > 10.20.7.2: ICMP echo request, id 222, seq 24, length 64

root@07_K1:~#
04, length 156
10:38:19.781143 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13475876:13475232, ack 7993, win 15
04, length 156
10:38:19.781280 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13475232:13475388, ack 7993, win 15
04, length 156
10:38:19.781259 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13475388:13475544, ack 7993, win 15
04, length 156
10:38:19.781311 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13475544:13475700, ack 7993, win 15
04, length 156
10:38:19.781373 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13475700:13475856, ack 7993, win 15
04, length 156
10:38:19.781435 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13475856:13476012, ack 7993, win 15
04, length 156
10:38:19.781522 IP 10.141.6.3.63468 > 172.22.7.2: Flags [.], ack 13466972, win 8253, length 0
10:38:19.781547 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13476168:13476324, ack 7993, win 15
04, length 156
10:38:19.781685 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13476324:13476480, ack 7993, win 15
04, length 156
10:38:19.781665 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13476480:13476632, ack 7993, win 15
04, length 252
10:38:19.781722 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13476632:13476888, ack 7993, win 15
04, length 156
10:38:19.781780 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13476888:13477044, ack 7993, win 15
04, length 156
10:38:19.781837 IP 172.22.7.2 > 10.141.6.3.63468: Flags [P.], seq 13477044:13477200, ack 7993, win 15
04, length 156

Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.8.0-85-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Oct 20 09:53:36 2025 from 10.141.6.3
root@07_K1:~# ping 10.20.7.2
PING 10.20.7.2 (10.20.7.2) 56(84) bytes of data.
^C
--- 10.20.7.2 ping statistics ---
56 packets transmitted, 0 received, 100% packet loss, time 56339ms

root@07_K1:~# route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 172.22.7.1 0.0.0.0 UG 0 0 0 eth0
10.10.7.0 0.0.0.0 255.255.255.0 U 0 0 0 eth1
172.22.7.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0

root@07_K1:~# ip r
default via 172.22.7.1 dev eth0
10.10.7.0/24 dev eth1 proto kernel scope link src 10.10.7.3
172.22.7.0/24 dev eth0 proto kernel scope link src 172.22.7.4
root@07_K1:~# ping 10.20.7.2
PING 10.20.7.2 (10.20.7.2) 56(84) bytes of data.
```

ping <adres IP> - wysłanie pinga na <adres IP>

3.4.1.11 Czy oprócz *ICMP echo request* są widoczne jeszcze jakieś inne ramki? Jeśli tak, jakie?

Odp: Tak - ARP, który wysyła zapytania na adresy IP aby uzyskać adresy MAC.

Nagłówek IP zawiera w sobie informację TTL (*Time to live*). TTL to licznik, który przy każdorazowym przejściu przez interfejs jest zmniejszany o 1. Gdy licznik spadnie do 0, pakiet jest odrzucany a

urządzenie, które taki pakiet odrzuciło, odsyła do źródła pakietu odpowiedni komunikat ICMP. Mechanizm ten pozwala ograniczyć negatywne skutki wystąpienia pętli routingu.

3.4.1.12 Jakiej wartości TTL jest domyślnie przesyłany w ping (*podpowiedź: taka informacja jest szczegółowa, tcpdump z flagą -v*). Udokumentuj zrzutem ekranu.

Odp: Domyślna wartość TTL, która jest przesyłana w ping wynosi 64.

Zrzut ekranu

The screenshot shows two terminal windows. The top window, titled 'root@07_K1: ~', displays the output of the 'route -n' command, showing the kernel IP routing table. It lists destinations 0.0.0.0, 10.10.7.0, and 172.22.7.0 with their respective gateways, netmasks, flags, metrics, and interfaces. Below this, it shows the output of 'ip r', displaying the default route and specific routes for 10.10.7.0/24 and 172.22.7.0/24. Finally, it shows the output of 'ping 10.20.7.2', indicating a 100% packet loss and a time of 377877ms. The bottom window, titled 'root@07_K3: ~', displays the output of 'tcpdump -i eth0 -s 0 -v icmp', showing a series of ICMP echo requests from 10.10.7.3 to 10.20.7.2, each with a TTL of 64.

```
root@07_K1: ~  
56 packets transmitted, 0 received, 100% packet loss, time 56339ms  
  
root@07_K1:~# route -n  
Kernel IP routing table  
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface  
0.0.0.0          172.22.7.1     0.0.0.0         UG    0      0      0 eth0  
10.10.7.0        0.0.0.0        255.255.255.0   U    0      0      0 eth1  
172.22.7.0       0.0.0.0        255.255.255.0   U    0      0      0 eth0  
  
root@07_K1:~# ip r  
default via 172.22.7.1 dev eth0  
10.10.7.0/24 dev eth1 proto kernel scope link src 10.10.7.3  
172.22.7.0/24 dev eth0 proto kernel scope link src 172.22.7.4  
root@07_K1:~# ping 10.20.7.2  
PING 10.20.7.2 (10.20.7.2) 56(84) bytes of data.  
exit  
^C  
--- 10.20.7.2 ping statistics ---  
370 packets transmitted, 0 received, 100% packet loss, time 377877ms  
  
root@07_K1:~# ping 10.20.7.2  
PING 10.20.7.2 (10.20.7.2) 56(84) bytes of data.  
|  
  
root@07_K3: ~  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 29, length 64  
10:44:38.941054 IP (tos 0x0, ttl 63, id 30280, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 30, length 64  
10:44:39.965078 IP (tos 0x0, ttl 63, id 30736, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 31, length 64  
10:44:40.989072 IP (tos 0x0, ttl 63, id 31554, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 32, length 64  
10:44:42.013034 IP (tos 0x0, ttl 63, id 32429, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 33, length 64  
10:44:43.037064 IP (tos 0x0, ttl 63, id 33179, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 34, length 64  
10:44:44.061058 IP (tos 0x0, ttl 63, id 34178, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 35, length 64  
10:44:45.085078 IP (tos 0x0, ttl 63, id 34612, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 36, length 64  
10:44:46.109041 IP (tos 0x0, ttl 63, id 35311, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 37, length 64  
10:44:47.133056 IP (tos 0x0, ttl 63, id 35861, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 38, length 64  
10:44:48.157051 IP (tos 0x0, ttl 63, id 36221, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 39, length 64  
10:44:49.181058 IP (tos 0x0, ttl 63, id 36620, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 40, length 64  
10:44:50.205089 IP (tos 0x0, ttl 63, id 36832, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 41, length 64  
10:44:51.229017 IP (tos 0x0, ttl 63, id 37567, offset 0, flags [DF], proto ICMP (1), length 84)  
10.10.7.3 > 10.20.7.2: ICMP echo request, id 230, seq 42, length 64
```

Jeśli ping przesyła się poprawnie, można zamknąć nadmiarowe terminale.

3.5 ARP

To zadanie ma na celu prezentację zachowania protokołu ARP.

Protokół ARP służy do określania jaka maszyna fizyczna (adres MAC) kryje się za adresem IP.

Należy zalogować się na K1, K2, K3, R

3.5.1.1 Sprawdzić na tych maszynach tablice ARP.

Zrzut ekranu – arp, 4 terminale

The screenshot shows four terminal windows arranged in a 2x2 grid. Each terminal is running on a root@10.148.0.131 machine (Ubuntu 20.04.6 LTS). The top-left terminal shows the output of the 'ip r' command, displaying the routing table. The top-right terminal shows the output of the 'route -n' command, displaying the kernel IP routing table. The bottom-left terminal shows the output of the 'arp -n' command, displaying the ARP table. The bottom-right terminal shows the output of the 'arp -n' command, displaying the ARP table. The output of the 'arp -n' command in the bottom-right terminal is as follows:

Address	HWtype	HWaddress	Flags	Mask	Iface
10.10.7.2	ether	1a:f4:3c:17:17:ac	C		eth2
172.22.7.1	ether	3e:ed:05:1a:ae:9d	C		eth0
10.10.7.3	ether	b6:48:26:7f:3b:a8	C		eth1

arp -n - tablica ARP

Należy otworzyć jeszcze jedno połączenie do K1, tak by w sumie widzieć 5 terminali (2 razy K1, oraz K2, K3, R). Na K1, K2, K3, R należy uruchomić nasłuchiwanie ruchu na interfejsie eth1

Na wolnym terminalu K1 należy uruchomić ping na adres 10.10.NN.100.

3.5.1.2 Jaki ruch jest obserwowany na K1, K2, K3, R? Udokumentować zrzutem ekranu.

Zrzut ekranu – 4 terminale

Ruch – zapytania ARP – jest obserwowany na eth1/K1, eth1/K2, eth1/R, a na eth1/K3 nie. Jest to spowodowane tym, że K1 i K2 są w tej samej podsieci, a K3 nie. Na routerze również pojawia się ruch, ponieważ w tym przypadku router działa także jako switch.

The screenshot displays four terminal windows from a network simulation. The top-left window (root@07_K1) shows a list of IP addresses and their corresponding MAC addresses, followed by a series of ARP requests. The top-right window (root@07_K3) shows a similar list of IP addresses and MAC addresses, but no ARP requests are visible. The bottom-left window (root@07_K2) shows a list of IP addresses and MAC addresses, followed by a series of ARP requests. The bottom-right window (root@07_R) shows a list of IP addresses and MAC addresses, but no ARP requests are visible. The terminal windows are arranged in a 2x2 grid, with the top row showing K1 and K3, and the bottom row showing K2 and R. The terminal windows are titled 'root@07_K1', 'root@07_K3', 'root@07_K2', and 'root@07_R'.

3.5.1.3 Dlaczego ARP request jest widoczny na K1, K2, R? Dlaczego nie jest widoczny na K3? Dlaczego nie obserwujemy ARP reply?

Odp: ARP Request jest widoczny na K1, K2 i R, ponieważ jest to komunikat broadcast ograniczony do ich wspólnego segmentu sieci (10.10.7.0/24). K3 go nie widzi, ponieważ router R nie przekazuje broadcastów do innej podsieci (10.20.7.0/24), skutecznie oddzielając domeny rozgłoszeniowe. ARP Reply nie jest obserwowany, gdyż jest to komunikat unicast (kierowany), a najczęściej oznacza to, że żaden host w segmencie nie posiadał poszukiwanego adresu IP i nie wygenerował odpowiedzi.

Można zakończyć nasłuchiwanie ruchu na K1, K2, K3, R

3.6 Aktywne połączenia

To zadanie ma na celu zapoznanie Studentów z komendą netstat.

Należy zestawić kilka (min. 3) połączeń SSH do K1 a następnie sprawdzić na nim listę aktywnych połączeń sieciowych. Podobną operację wykonać dla K2.

```
ss -tupn
```

3.6.1.1 Udokumentować:

Zrzut ekranu (lista aktywnych połączeń K1)

[illegible]

Zrzut ekranu (lista aktywnych połączeń K2)

[illegible]

Po wykonaniu rzutów ekranu można zamknąć nadmiarowe połączenia SSH.

3.6.1.2 Proszę uzupełnić tekst:

Obserwowane w sekcjach Local/Peer Address adresy składają się z dwóch części oddzielonych znakiem „:”. Pierwsza część to **adres IP**, druga to **numer portu**. Przykładowo, serwer SSH na K1 oraz K2 nasłuchuje na tym samym **porcie** nr **22**, dlatego w sekcji Local Address widnieją wpisy **172.22.7.4:22** i **172.22.7.5:22** (podać dla dwóch Komputerów). Z serwerem SSH na K1 nawiązane są **3** połączenia. Wartości Peer Address identyfikują klientów SSH – ich **IP** i używane przez nich **porty**.

3.6.1.3 Dlaczego serwer SSH jest zawsze* uruchamiany na tym samym porcie, natomiast poszczególni klienci SSH korzystają z losowych (wysokich) numerów portów? (podpowiedź: kto inicjuje połączenie, jakie informacje są przesyłane w nagłówku IP? Także https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers)

Odp: Serwer SSH jest uruchamiany na stałym, znanym porcie (domyślnie 22) jako Port Docelowy w komunikacji TCP/IP, co jest konieczne, aby klienci mogli łatwo zlokalizować i zainicjować połączenie z usługą. Klienci SSH natomiast korzystają z losowych, wysokich portów jako Port Źródłowy, aby zapewnić unikatową identyfikację każdej sesji na swoim komputerze i uniknąć kolizji z portami zarezerwowanymi dla innych standardowych usług. Różne role portów (docelowy vs. źródłowy) pozwalają na bezbłędne mapowanie ruchu sieciowego w obie strony.

**serwer SSH można uruchomić na dowolnym porcie, ale wymaga to zmiany domyślnej konfiguracji. W przypadku użycia portu innego niż domyślny, klient musi podać ten port (flaga -p).*

Proszę zauważyć, że łączymy się na adres 10.140.0.131 i porty 2NN11, 2NN12, ..., i także korzystamy z flagi -p. Wszystkie kontenery mają uruchomione serwery SSH na porcie 22. Na serwerze 10.140.0.131 ustawione jest przekierowanie portów, tzn. dane kierowane na adres np. 10.140.0.131:2NN11 są przesyłane na adres 172.22.NN.XX:22 (XX jest przydzielane automatycznie przez program docker). Mechanizm przekierowania portów nie będzie badany w ramach tego ćwiczenia.

3.7 Zmiana adresu IP (awaria i odzyskiwanie)

To zadanie ma na celu wywołanie awarii dostępu spowodowanej błędną konfiguracją interfejsu sieciowego. Następnie zaprezentowany zostanie sposób odzyskania kontroli nad sytuacją.

3.7.1.1 Należy sprawdzić konfigurację adresów sieciowych na K4.

Zrzut ekranu

```
root@07_K4:~# ip -br a
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0@if1121       UP                172.22.7.3/24
eth1@if1126       UP                10.20.7.3/24
root@07_K4:~# |
```

`ip -br a` – wypisuje informacje o aktywnych interfejsach

3.7.1.2 Proszę zmienić adres K4/eth1 na 10.20.NN.100/8. Żeby to zrobić, należy usunąć aktualny adres z interfejsu eth1 i dodać na nim nowy adres (`ip a d ... / ip a a ...`). Po dodaniu adresu z maską /8 utracimy komunikację z K4.

Zrzut ekranu – brak połączenia

```
root@07_K4: ~
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@07_K4:~# ip -br a
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0@if1121       UP                172.22.7.3/24
eth1@if1126       UP                10.20.7.3/24
root@07_K4:~# ip a d 10.20.7.3/24 dev eth1
root@07_K4:~# ip -br a
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0@if1121       UP                172.22.7.3/24
eth1@if1126       UP
root@07_K4:~# ip a a 10.20.07.100/8 eth1
Error: either "local" is duplicate, or "eth1" is a garbage.
root@07_K4:~# ip a a 10.20.07.100/8 dev eth1
|
```

Po ustawieniu adresu w ten sposób powinna zostać utracona łączność z K4. Kolejne próby łączenia się na adres 10.140.0.131 -p 2NN14 się nie udają. Studenci przypominają sobie jednak, że wszystkie kontenery są podłączone do sieci *Lab_control_SC* (interfejsami eth0).

Należy z któregośkolwiek Komputera lub Rutera (K1, K2, K3, R) zalogować się przez SSH na adres K4/eth0.

```
ssh root@172.22.NN.X
```

X należy odczytać z wykonanego na początku tego zadania zrzutu ekranu konfiguracji adresów sieciowych na K4

3.7.1.3 Po zalogowaniu na K4 należy sprawdzić konfigurację adresów sieciowych na K4.

Zrzut ekranu - konfiguracja interfejsów

```
root@07_K4: ~  
* Management: https://landscape.canonical.com  
* Support: https://ubuntu.com/pro  
  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
Last login: Mon Oct 20 11:04:33 2025 from 10.141.6.3  
root@07_K4:~# pi -br -a  
-bash: pi: command not found  
root@07_K4:~# ip -br -a  
Usage: ip [ OPTIONS ] OBJECT { COMMAND | help }  
ip [ -force ] -batch filename  
where OBJECT := { link | address | addrlabel | route | rule | neigh | ntable |  
tunnel | tuntap | maddress | mroute | mrule | monitor | xfrm |  
netns | l2tp | fou | macsec | tcp_metrics | token | netconf | ila |  
vrf | sr | nexthop }  
OPTIONS := { -V[ersion] | -s[tatistics] | -d[etails] | -r[esolve] |  
-h[uman-readable] | -i[c] | -j[son] | -p[retty] |  
-f[amily] { inet | inet6 | mpls | bridge | link } |  
-4 | -6 | -I | -D | -M | -B | -O |  
-l[oops] { maximum-addr-flush-attempts } | -br[ief] |  
-o[neline] | -t[imestamp] | -ts[hort] | -b[atch] [filename] |  
-rc[vbuf] [size] | -n[etns] name | -N[umeric] | -a[ll] |  
-c[olor]}  
root@07_K4:~# ip -br a  
lo UNKNOWN 127.0.0.1/8 ::1/128  
eth0@if1121 UP 172.22.7.3/24  
eth1@if1126 UP 10.20.7.100/8  
root@07_K4:~# D
```

3.7.1.4 Należy sprawdzić tablicę routingu na K4

Zrzut ekranu

```
root@07_K4:~# route -n  
Kernel IP routing table  
Destination Gateway Genmask Flags Metric Ref Use Iface  
0.0.0.0 172.22.7.1 0.0.0.0 UG 0 0 0 eth0  
10.0.0.0 0.0.0.0 255.0.0.0 U 0 0 0 eth1  
172.22.7.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0  
root@07_K4:~# |
```

3.7.1.5 Na podstawie tablicy routingu należy odpowiedzieć, gdzie aktualnie kierowany jest ruch adresowany do Komputera Studenta? (Podpowiedź: Jaki adres ma Komputer Studenta? Pod który wpis w tablicy routingu podpada?)

Odp: Ruch do Komputera Studenta trafia do interfejsu **eth1**, ponieważ jego adres IP (10.141.6.3) mieści się w sieci 10.0.0.0/8, która w tablicy routingu jest przypisana do interfejsu eth1 w K4 bez użycia bramy.

```

root@07_K4:~# route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          172.22.7.1     0.0.0.0         UG    0      0      0 eth0
10.0.0.0         0.0.0.0        255.0.0.0       U     0      0      0 eth1
172.22.7.0       0.0.0.0        255.255.255.0   U     0      0      0 eth0

```

Należy we właściwy sposób ustawić adres 10.20.NN.100/24 na K4/eth1.

3.7.1.6 Należy udokumentować: Możliwość połączenia do K4 z Komputera Studenta, konfigurację K4/eth1, tablicę routingu na K4.

Zrzut ekranu – SSH na K4

```

root@07_K4:~# ss -tupn
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port      Process
tcp        ESTAB      0           0           172.22.7.3:22           172.22.7.1:39442       users:(("ssh
d",pid=74,fid=4))
root@07_K4:~# |

```

Zrzut ekranu - konfiguracja K4/eth1

```

root@07_K4:~# ip route add 10.128.0.0/9 via 10.20.7.100 dev eth1

```

Zrzut ekranu – tablica routingu K4

```

root@07_K4:~# route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          172.22.7.1     0.0.0.0         UG    0      0      0 eth0
10.0.0.0         0.0.0.0        255.0.0.0       U     0      0      0 eth1
10.128.0.0       10.20.7.100    255.128.0.0     UG    0      0      0 eth1
172.22.7.0       0.0.0.0        255.255.255.0   U     0      0      0 eth0
root@07_K4:~# |

```